

Chi tiết: yếu tố không thể xem nhẹ

Li Ruizhe

Nguồn gốc: Details: an element that cannot be neglected

IOI China candidate team papers / 2004 / Li Ruizhe.pdf

Tóm tắt

Một thuật toán dù tốt đến đâu, nếu xử lý chi tiết không đúng, cũng có thể trở thành một “thuật toán rác”. Khi chú ý đến cách cài đặt ở mức tổng thể, người lập trình lại thường bỏ qua những vấn đề nhỏ. Bài viết nêu ảnh hưởng của chất lượng xử lý chi tiết lên cài đặt thuật toán, rồi xét hai mặt khác nhau của ảnh hưởng ấy đối với độ phức tạp thời gian. Ba ví dụ điển hình được dùng để nhấn mạnh yếu tố thường bị xem nhẹ này.

Từ khóa. Chi tiết; độ phức tạp thời gian; thuật toán.

Contents

1	Dẫn nhập	1
2	Chi tiết hầu như không ảnh hưởng đến bậc độ phức tạp	2
2.1	Ví dụ 1: duy trì danh sách tuyến tính	2
3	Chi tiết ảnh hưởng đến độ phức tạp thời gian	4
3.1	Ví dụ 2: Truyền thuyết anh hùng ngân hà	4
3.2	Ví dụ 3: Đường sắt	6
4	Kết luận	8

1 Dẫn nhập

Trong lập trình, thuật toán là phần cốt lõi. Một thuật toán tốt thường đem lại hiệu quả cao hơn nhiều so với các thuật toán khác. Nhưng khi theo đuổi thuật toán tốt hơn, ta lại dễ bỏ qua những chi tiết trong thuật toán.

Nhiều khi ta tự cho rằng mình đã dùng thuật toán tối ưu, nhưng kết quả vẫn không như mong muốn. Nguyên nhân là gì? Chính là chi tiết. Một số thiếu sót nhỏ ở chi tiết thường làm cho những chỗ then chốt của thuật toán chạy chậm, thậm chí làm độ phức tạp thời gian cao hơn rất nhiều so với bình thường. Trong trường hợp nghiêm trọng nhất, chúng còn làm cả thuật toán sai.

Vì vậy, chi tiết có vai trò rất quan trọng trong lập trình. Mượn cách nói của triết học, chi tiết là bộ phận then chốt của chỉnh thể thuật toán, và bộ phận then chốt này thường quyết định trạng thái hiệu năng của toàn bộ chỉnh thể. Theo tính chất và mức độ ảnh hưởng lên thuật toán, có thể chia chi tiết thành ba loại:

1. Những chi tiết hầu như không ảnh hưởng đến bậc độ phức tạp thời gian. Chúng không tác động căn bản đến độ phức tạp, mà chỉ làm thay đổi hệ số.
2. Những chi tiết ảnh hưởng đến độ phức tạp thời gian. Loại chi tiết này khá kín đáo, thường không được chú ý. Tuy nhiên ảnh hưởng của nó rất lớn: xử lý tốt hay không tốt có thể tạo ra khác biệt về bản chất trong hiệu quả thời gian của chương trình.
3. Những chi tiết ảnh hưởng đến tính đúng sai của thuật toán. Loại này có ảnh hưởng lớn nhất. Trong thi đấu, nhiều thí sinh đã tìm được hướng giải nhưng vẫn không qua hết dữ liệu kiểm thử, thường vì xử lý những chi tiết này không thỏa đáng.

Loại thứ ba chắc hẳn ai cũng từng cảm nhận sâu sắc; nếu bàn riêng thì sẽ rất dài. Vì vậy phần dưới chủ yếu phân tích hai loại đầu.

2 Chi tiết hầu như không ảnh hưởng đến bậc độ phức tạp

Loại chi tiết này không ảnh hưởng quá lớn đến thuật toán. Tuy nhiên ở thời điểm then chốt, hệ số trong độ phức tạp thời gian vẫn có tác động rõ rệt đến hiệu quả. Chẳng hạn với một chi tiết nào đó, cách xử lý A có thời gian $O(n)$, còn cách xử lý B có thời gian $O(2n)$; ở đây ta ghi hệ số vào biểu thức độ phức tạp chỉ để tiện diễn đạt. Nếu dùng A thì toàn bộ thuật toán mất 1 giây, còn dùng B sẽ mất 2 giây. Do đó, ta vẫn nên tối ưu cách xử lý chi tiết để hệ số của độ phức tạp được hạ xuống mức thấp hơn.

Ta xét ví dụ sau.

2.1 Ví dụ 1: duy trì danh sách tuyến tính

Cho một danh sách tuyến tính có kiểu dữ liệu là chuỗi, cùng một số quy tắc thao tác bảo trì. Hãy viết chương trình mô phỏng các thao tác trên danh sách đó.

Định nghĩa một con trỏ động, trỏ tới phần tử danh sách đang được xử lý. Có bốn loại thao tác:

1. Chèn: chèn dữ liệu vào cuối danh sách.
2. Di chuyển con trỏ: dời con trỏ động về sau một số vị trí.
3. Xóa: xóa một số phần tử bắt đầu từ phần tử mà con trỏ động đang trỏ tới.
4. In: in phần tử ở vị trí mà con trỏ động đang trỏ tới.

Để tiện cài đặt, chương trình đọc từ `line.in` và ghi kết quả vào `line.out`. Quy mô dữ liệu vào không cố định; định dạng của từng thao tác trong tập tin vào như sau:

1. Chèn gồm hai dòng, dòng đầu là ADD, dòng thứ hai là dữ liệu cần chèn, ví dụ:

```
ADD
abcdef
```

2. Di chuyển con trỏ gồm hai dòng, dòng đầu là MOVE, dòng thứ hai là số đơn vị cần di chuyển, ví dụ:

```
MOVE
2
```

3. Xóa gồm hai dòng, dòng đầu là DEL, dòng thứ hai là số phần tử cần xóa, ví dụ:

```
DEL
3
```

4. In gồm một dòng PRINT.

Mô hình của bài toán rất đơn giản. Vì cần tăng, xóa phần tử động và không có giới hạn không gian cố định, ta nên cài đặt bằng con trỏ động. Cấu trúc dữ liệu Pascal tương ứng cũng rất đơn giản:

```
PNode=^TNode;
TNode=Record
    Data:String;
    Next:PNode;
End;
```

Các thao tác chèn, di chuyển và in không cần bàn kỹ. Điểm đáng xem xét là thao tác xóa. Vì các thao tác có thể chèn và xóa phần tử thường xuyên, ta không thể để mặc những phần tử đã xóa mà không xử lý; nếu làm vậy, chẳng bao lâu các phần tử bị bỏ đi sẽ chiếm hết bộ nhớ. Do đó khi xóa, ta phải giải phóng bộ nhớ.

Một cách làm tự nhiên là mỗi khi xóa phần tử, ta giải phóng không gian của từng phần tử bị xóa; khi về sau cần chèn phần tử thì cấp phát lại.

Cách cài đặt này giải quyết được vấn đề không giải phóng không gian, nhưng lại sinh ra vấn đề khác: cấp phát và giải phóng bộ nhớ quá thường xuyên. Thao tác giải phóng không tốn nhiều thời gian, nhưng cấp phát bộ nhớ thì không hề rẻ.

Cấp phát bộ nhớ không đơn giản như một lệnh new trong chương trình. Ở tầng thấp của hệ thống, quá trình ấy cần một loạt thao tác. Trước hết hệ thống phải tìm trong bộ nhớ một khối đủ lớn để cấp phát. Vì bộ nhớ đã dùng không nhất thiết liên tục và có thể bị phân mảnh, thời gian tìm kiếm cũng rất đáng kể. Giả sử thời gian trung bình cho một lần cấp phát là $O(h)$, thì n lần cấp phát có độ phức tạp $O(hn)$, không phải $O(n)$ như ta thường tưởng.

Vì vậy, nếu muốn tăng hiệu quả thao tác trên danh sách liên kết, đây là một điểm đột phá tốt. Xóa phần tử thì phải giải phóng bộ nhớ, thêm phần tử thì phải cấp phát bộ nhớ; liệu có thể dùng trực tiếp không gian của phần tử sắp xóa cho thao tác thêm phần tử, thay vì cấp phát lại hay không? Câu trả lời là có.

Gọi danh sách tuyến tính đang dùng là **danh sách gốc**. Ta tạo thêm một danh sách liên kết khác, gọi là **danh sách thu hồi**. Với mỗi lần xóa, nhờ tính linh hoạt của con trỏ, ta có thể tách trực tiếp đoạn bị xóa khỏi danh sách gốc rồi nối nó vào cuối danh sách thu hồi, thay cho việc giải phóng ngay. Khi chèn, trước hết kiểm tra danh sách thu hồi có rỗng không. Nếu không rỗng thì lấy ngay một nút từ đó để nối vào danh sách gốc; nếu rỗng mới xin hệ thống cấp phát bộ nhớ.

```
Ban đầu:  Data1  Data2  Data3  Data4  Data5
           ↓ xóa Data2, Data3
Gốc:      Data1  Data4  Data5
Thu hồi:   Data2  Data3
           ↓ thêm Data6, dùng nút Data2
Gốc:      Data1  Data4  Data5  Data6
Thu hồi:   Data3
```

Sau tối ưu này, số lần thực sự cần yêu cầu hệ thống cấp phát bộ nhớ đúng bằng số nút lớn nhất từng

xuất hiện trong toàn bộ quá trình bảo trì. Trong tình huống thông thường, khi thêm nút ta lấy không gian từ danh sách thu hồi, nên n thao tác thêm chỉ có độ phức tạp $O(n)$.

Kết quả thử nghiệm thực tế như sau:

Điểm thử	Thuật toán gốc	Thuật toán tối ưu
1	2.149s	2.012s
2	6.588s	5.672s
3	13.202s	12.924s
4	5.086s	4.911s
5	10.297s	9.918s

Môi trường thử nghiệm là AMD Duron 1.1GHz, 256MB SDRAM, Debian Linux với kernel 2.6.0; hệ thống chấm là Celiz 0.0.3.

Từ kết quả trên có thể thấy thuật toán sau tối ưu có lợi thế nhất định so với thuật toán gốc. Lợi thế ấy không quá lớn, nhưng việc tăng hiệu quả là rõ ràng. Như vậy, dù loại chi tiết này không ảnh hưởng nhiều đến bậc độ phức tạp thời gian, xử lý tốt nó vẫn đem lại ích lợi.

3 Chi tiết ảnh hưởng đến độ phức tạp thời gian

So với loại chi tiết ở trên, loại này ảnh hưởng đáng kể đến hiệu quả thời gian của thuật toán. Khi xử lý chúng, ta lại thường dễ đi vào cách làm không thích hợp, khiến độ phức tạp tăng lên. Vì vậy, xử lý tốt loại chi tiết này là điểm mấu chốt để thuật toán giải quyết bài toán một cách hiệu quả.

Trước hết ta xem một ví dụ về cách một thuật toán rất quen thuộc xử lý chi tiết đúng chỗ.

3.1 Ví dụ 2: Truyền thuyết anh hùng ngân hà

Bài toán này là câu 1, vòng 1 của NOI 2002.

Mô tả bài toán. Năm 5801 công nguyên, cư dân Trái Đất di cư tới hành tinh thứ hai của sao Alpha trong chòm Kim Ngưu, tuyên bố thành lập Liên bang Ngân hà. Cùng năm đó được đổi thành năm đầu của lịch vũ trụ, và loài người bắt đầu mở rộng vào sâu trong dải Ngân hà.

Năm 799 lịch vũ trụ, hai tập đoàn quân sự lớn của Ngân hà giao chiến tại vùng sao Bamilion. Phe Taishanyading cử tư lệnh hạm đội Reinhard chỉ huy hơn mười vạn chiến hạm xuất trận; phe Qitunshanhe chỉ định Yang Wen-li tổ chức ba vạn chiến hạm dưới quyền để nghênh chiến.

Yang Wen-li giỏi bày binh bố trận, nhiều lần dùng chiến thuật khéo léo để lấy ít thắng nhiều, nên khó tránh khỏi sinh lòng kiêu ngạo. Trong trận quyết chiến này, ông chia chiến trường Bamilion thành 30000 cột, đánh số $1, 2, \dots, 30000$. Sau đó ông cũng đánh số các chiến hạm của mình là $1, 2, \dots, 30000$, đặt chiến hạm số i ở cột i , tạo thành trận hình ban đầu dạng một hàng dài. Khi địch tiến đến, Yang Wen-li nhiều lần phát lệnh hợp nhất để tập trung phần lớn chiến hạm vào một số cột và tấn công dày đặc.

Lệnh hợp nhất có dạng $M \ i \ j$: cả đội chiến hạm chứa chiến hạm số i , giữ nguyên thứ tự từ đầu đến cuối, được nối vào cuối đội chiến hạm chứa chiến hạm số j . Mỗi đội chiến hạm gồm một hoặc nhiều chiến hạm nằm trong cùng một cột. Sau mỗi lệnh hợp nhất, đội ở cột đích lớn hơn.

Tuy nhiên, Reinhard lão luyện đã giành thế chủ động về chiến lược. Trong giao chiến, ông có thể thông qua mạng tình báo khổng lồ để nghe lén mọi lệnh điều động hạm đội của Yang Wen-li. Đồng thời, để nắm kịp phân bố chiến hạm hiện tại, Reinhard cũng phát ra các truy vấn $C \ i \ j$. Truy vấn này hỏi máy tính xem chiến hạm số i và chiến hạm số j hiện có ở cùng một cột hay không; nếu có, giữa chúng có bao nhiêu chiến hạm.

Bạn cần viết chương trình phân tích các lệnh của Yang Wen-li và trả lời các truy vấn của Reinhard.

Dữ liệu vào. Tập tin `galaxy.in` có dòng đầu là số nguyên T ($1 \leq T \leq 500000$), số lệnh. Tiếp theo là T dòng, mỗi dòng là một lệnh:

- $M \ i \ j$: i, j là các số nguyên ($1 \leq i, j \leq 30000$), bảo đảm chiến hạm i và j không ở cùng một cột.
- $C \ i \ j$: i, j là các số nguyên ($1 \leq i, j \leq 30000$), biểu diễn một truy vấn.

Dữ liệu ra. Ghi ra `galaxy.out`. Với lệnh điều động M , chương trình chỉ cập nhật trạng thái và không in gì. Với truy vấn C , in một dòng chứa số chiến hạm nằm giữa chiến hạm i và chiến hạm j nếu chúng ở cùng một cột; nếu không, in -1 .

Ví dụ.

Input

4

M 2 3

C 1 2

M 2 4

C 4 2

Output

-1

1

Sau lệnh $M \ 2 \ 3$, chiến hạm 2 được nối sau chiến hạm 3, còn chiến hạm 1 vẫn ở cột riêng nên truy vấn $C \ 1 \ 2$ cho kết quả -1 . Sau lệnh $M \ 2 \ 4$, đội chứa 2 và 3 được nối vào sau chiến hạm 4; giữa 4 và 2 chỉ có chiến hạm 3, nên truy vấn cuối in 1.

Phân tích. Mỗi cột có thể xem là một tập hợp. Khi đó hợp nhất hạm đội và truy vấn cùng cột chính là thao tác hợp và tìm trên các tập hợp, một mô hình rất điển hình của cấu trúc **hợp nhất–tìm kiếm** (disjoint set union).

Tuy nhiên, cấu trúc này nếu dùng đơn thuần vẫn có khuyết điểm: thao tác hợp có thể làm cây gồm n nút suy biến thành một dây chuyền, ảnh hưởng lớn đến hiệu quả truy vấn. Vì vậy cần tối ưu. Hai cách tối ưu thường dùng là hợp cây nhỏ vào cây lớn và nén đường đi.

Cách cài đặt cụ thể của cấu trúc hợp nhất–tìm kiếm và hai tối ưu trên đã được trình bày nhiều trong sách, nên không nhắc lại. Các tài liệu đều nói rằng nén đường đi được thực hiện trong quá trình tìm kiếm. Ta có thể tự hỏi: vì sao lại nén khi tìm, mà không nén trong quá trình hợp?

Hãy phân tích độ phức tạp của hai thao tác hợp và tìm. Gọi h là độ sâu từ nút cần tìm đến gốc. Thao tác tìm có độ phức tạp $O(h)$. Thao tác hợp bản thân có độ phức tạp $O(1)$, nhưng trước khi hợp ta thường phải tìm gốc của hai tập hợp cần hợp, như trong bài này, nên thời gian cũng thường là $O(h)$. Trong thao tác hợp không có chỗ tối ưu đặc biệt; nén đường đi thực chất là làm giảm h trong thao tác tìm, từ đó giảm thời gian tìm kiếm.

Nếu nén đường đi trong khi tìm, thì một truy vấn đi từ nút cần tìm lên gốc mất $O(h)$. Sau khi biết gốc, ta chỉ cần đổi cha của mọi nút trên đường đi sang gốc, cũng mất $O(h)$. Do đó tìm cộng với nén đường đi vẫn là $O(h)$, chỉ tăng hệ số lên khoảng hai lần. Thao tác hợp cũng là $O(h)$. Hơn nữa, có thể

chứng minh rằng khi dùng tối ưu hợp cây nhỏ vào cây lớn, độ sâu mỗi cây không vượt quá $\log n$, với n là số nút trong cây; vì vậy thời gian tìm và hợp khi ấy là $O(\log n)$.

Nếu nén đường đi trong khi hợp, thì mọi nút của cây bị hợp phải đổi cha sang gốc của cây đích. Quá trình này có độ phức tạp $O(n)$, trong đó n là số nút của cây bị hợp. Đổi lại, sau thao tác ấy, mỗi cây chỉ có độ sâu không quá 2, nên truy vấn tìm có độ phức tạp $O(1)$.

Cách xử lý	Tìm	Hợp
Nén đường đi khi tìm	$O(\log n)$	$O(\log n)$
Nén đường đi khi hợp	$O(1)$	$O(n)$

Nhìn riêng bảng trên, hai cách dường như đều có lợi thế: nén trong hợp có vẻ tốt khi có rất nhiều truy vấn tìm. Nhưng với cách nén khi tìm, thời gian không phải lúc nào cũng là $O(\log n)$. Khi số lần tìm tăng lên, các đường đi liên tục được nén, nên thời gian thực tế cũng giảm. Người ta đã chứng minh rằng n thao tác tìm chỉ cần nhiều nhất $O(n\alpha(n))$ thời gian, trong đó $\alpha(n)$ là hàm ngược Ackermann một biến, tăng chậm hơn rất nhiều so với $\log n$ nhưng không phải hằng số. Với các số nguyên dương thường gặp, $\alpha(n) \leq 4$, nên thao tác tìm thực tế đã rất gần $O(1)$. Vì vậy ở mặt truy vấn, nén khi tìm không yếu hơn đáng kể so với nén khi hợp.

Trong thao tác hợp, ngược lại, $O(\log n)$ của cách nén khi tìm rõ ràng có lợi thế lớn so với $O(n)$ của cách nén khi hợp. Qua lập luận trên, ta có thể kết luận rằng nén đường đi trong thao tác tìm có hiệu quả thời gian cao hơn nén đường đi trong thao tác hợp.

A								A
B	C		D					B
E	F	G		H				C
I	J							D
								E
								F
								G
								H
								I
								J

Một cây hợp nhất–tìm kiếm ban đầu
sau khi tìm I, đường đi $I \rightarrow A$ được nén

Ta xét tiếp một ví dụ khác, nơi một chi tiết xử lý khác nhau tạo ra hiệu quả hoàn toàn khác nhau.

3.2 Ví dụ 3: Đường sắt

Bộ đường sắt bang Byteotia quyết định bắt kịp thời đại bằng cách đưa vào hệ thống nối mạng giữa các thành phố. Do thiếu máy móc hiệu quả, toa xe sạch và đường ray thẳng, họ chỉ có thể xây dựng một mạng đơn giản như sau. Việc thiếu hệ thống đặt chỗ bằng máy tính là một trở ngại khác. Nhiệm vụ của bạn là viết phần chính của hệ thống đó.

Để đơn giản, giả sử mạng thành phố nối tuần tự c thành phố, đánh số từ 1 đến c ; thành phố đầu là 1, thành phố cuối là c . Mỗi tàu có s chỗ, và trên mọi đoạn giữa hai ga liên tiếp, tàu không được chở quá số chỗ này.

Hệ thống máy tính nhận liên tiếp các yêu cầu đặt chỗ và quyết định có chấp nhận hay không. Nếu tàu còn đủ chỗ trên toàn bộ đoạn được yêu cầu thì yêu cầu được chấp nhận, ngược lại bị từ chối. Không được chấp nhận một phần yêu cầu, chẳng hạn chỉ chấp nhận một đoạn đường hoặc một phần số hành khách. Sau khi chấp nhận, số ghế trống trong tàu phải được cập nhật. Các yêu cầu được xử lý theo đúng thứ tự nhận được.

Hãy viết chương trình đọc mạng đường sắt và danh sách yêu cầu từ `kol.in`, tính yêu cầu nào được chấp nhận hoặc từ chối, rồi ghi kết quả ra `kol.out`.

Dữ liệu vào. Dòng đầu gồm ba số nguyên c, s, r ($1 \leq c \leq 60000$, $1 \leq s \leq 60000$, $1 \leq r \leq 60000$): số thành phố, số ghế trên tàu và số yêu cầu. Tiếp theo là r dòng; dòng thứ $i + 1$ mô tả yêu cầu thứ i bằng ba số nguyên o, d, n ($1 \leq o < d \leq c$, $1 \leq n \leq s$), lần lượt là ga xuất phát, ga đích và số ghế cần đặt.

Dữ liệu ra. In r dòng. Dòng thứ i là ký tự T nếu yêu cầu thứ i được chấp nhận, và N nếu không.

Ví dụ.

Input

4 6 4

1 4 2

1 3 2

2 4 3

1 2 3

Output

T

T

N

N

Phân tích. Bài toán yêu cầu cài đặt một hệ thống truy vấn gồm hai thao tác: hỏi và sửa. Nếu dùng danh sách tuyến tính thông thường, gọi n là khoảng cách giữa ga đi và ga đến trong truy vấn, thì mỗi lần hỏi mất $O(n)$ và mỗi lần sửa cũng mất $O(n)$. Trong bài này n có thể đạt 60000, số thao tác cũng lớn, nên cách đó không khả thi.

Ta có thể dùng cây đoạn. Sau khi chia toàn bộ tuyến đường, lưu các đoạn theo tư tưởng cây nhị phân. Ví dụ:

```

                        [1,9] 3
                [1,5] 3                [5,9] 4
        [1,3] 4        [3,5] 3        [5,7] 4        [7,9] 5
    [1,2] 5  [2,3] 4  [3,4] 3  [4,5] 3  [5,6] 4  [6,7] 5  [7,8] 5  [8,9] 5
```

Trong đó “[1, 9]” biểu diễn đoạn giữa ga 1 và ga 9; số 3 phía sau là số ghế trống nhỏ nhất trên mọi đoạn con của đoạn này, dưới đây gọi ngắn là **số ghế trống** của đoạn. Ví dụ, đoạn [1, 2] còn 5 ghế, đoạn [2, 3] còn 4 ghế, nên số ghế trống của [1, 3] là giá trị nhỏ hơn, tức 4; điều này cho biết trên toàn bộ đoạn [1, 3] chỉ có thể nhận nhiều nhất 4 ghế.

Với thao tác hỏi, bắt đầu từ gốc cây, ta liên tục chia đoạn truy vấn rồi đi đệ quy vào các con cho tới khi đoạn cần hỏi khớp với đoạn của một nút; khi đó chỉ cần kiểm tra số ghế trống của đoạn ấy có đáp ứng yêu cầu hay không. Chẳng hạn khi hỏi đoạn [4, 7], các nút như [4, 5] và [5, 7] là những nút mà ta không cần xét tiếp con của chúng; tạm gọi chúng là **nút đáy**. Để chứng minh mỗi lần hỏi có nhiều nhất bốn nút đáy, và vì độ sâu cây là $\log n$, nên thời gian hỏi là $O(\log n)$.

Với thao tác sửa, nếu sửa tất cả thông tin cần thay đổi, chẳng hạn giảm số ghế của đoạn [4, 7] đi 3, số nút phải cập nhật sẽ lớn hơn nhiều số đoạn cơ bản thật sự bị ảnh hưởng. Trong ví dụ này, [4, 7] chỉ gồm ba đoạn cơ bản, nhưng nếu cập nhật cả các nút tổ tiên thì phải đụng tới nhiều nút hơn. Nếu đoạn sửa rộng hơn, cách làm này còn có thể kém hơn danh sách tuyến tính.

Sai lầm ở đây là rơi vào quán tính: với các nút như [5, 6] và [6, 7], ta cập nhật chúng ngay, nhưng về sau có thể không bao giờ truy cập lại. Khi đó thao tác trên chúng hoàn toàn không có ý nghĩa.

Đổi góc nhìn, thật ra không cần sửa mọi nút ngay. Vì cây đoạn là một cây nhị phân, ta có thể ghi lại phần cần sửa và chỉ đẩy nó xuống khi thật sự cần. Ví dụ cần giảm 2 ghế trên đoạn [4, 9]. Khi sửa tới nút đáy [5, 9], theo cách cũ ta sẽ sửa toàn bộ các con và hậu duệ của [5, 9]. Thay vào đó, ta không sửa các nút ấy ngay, mà gán cho hai con của [5, 9] một dấu hiệu hiệu chỉnh -2 , như thể “báo” cho chúng rằng số ghế trống cần được giảm thêm 2.

Khi lần truy vấn sau đi qua một nút có dấu hiệu hiệu chỉnh, chẳng hạn hỏi [5, 7], ta mới dùng giá trị hiệu chỉnh để sửa số ghế trống của nút đó, rồi truyền giá trị hiệu chỉnh xuống các con. Nhờ vậy, các nút không còn được hỏi tới sẽ không phải sửa. Việc sửa tại một nút và truyền dấu hiệu xuống con chỉ mất $O(1)$, nên hiệu quả hỏi không bị ảnh hưởng. Sau tối ưu này, mỗi lần sửa cũng chỉ đi đến các nút đáy rồi dừng, do đó độ phức tạp sửa giảm xuống $O(\log n)$.

Từ ví dụ trên có thể thấy một số chi tiết tưởng như nhỏ lại làm độ phức tạp của thuật toán đi theo hai hướng hoàn toàn khác nhau. Vì vậy khi chú ý đến thuật toán, ta cũng không nên bỏ qua cách xử lý chi tiết; cần cân nhắc kỹ mỗi chi tiết nên được xử lý thế nào để không phạm lỗi và ảnh hưởng đến toàn bộ cài đặt.

4 Kết luận

Các chi tiết then chốt giữ vai trò rất quan trọng đối với toàn bộ thuật toán. Như đã trình bày ở trên, xử lý chi tiết tốt hay xấu ảnh hưởng trực tiếp đến tính đúng đắn; ngay cả khi không ảnh hưởng đến tính đúng đắn, nó vẫn ít nhiều tác động đến hiệu quả thời gian. Ta không nên sơ suất trong việc xử lý chi tiết, nếu không sẽ phải trả giá lớn hơn lợi ích thu được.

Tài liệu tham khảo

1. Wang Xiaodong, *Cấu trúc dữ liệu và thiết kế thuật toán*, Nhà xuất bản Công nghiệp Điện tử.
2. Đề vòng 1 của NOI 2002.

Chương trình tham khảo

Ví dụ 1: duy trì danh sách tuyến tính

Thuật toán chưa tối ưu.

```

Program Line_A;

Type
  PNode=^TNode;
  TNode=Record
    Data:String;
    Next:PNode;
  End;

Var
  Head,Now,Ending:PNode;

Procedure Init;
Begin
  Assign(Input,'line.in');
  Reset(Input);
  Assign(Output,'line.out');
  Rewrite(Output);
  New(Head);
  FillChar(Head^,SizeOf(Head^),0);
  Now:=Head;
  Ending:=Head;

```



```

End;

Procedure DoAdd;
Var
    tmp:PNode;
    st:String;
Begin
    Readln(st);
    New(tmp);
    tmp^.Data:=st;
    Ending^.Next:=tmp;
    tmp^.Next:=nil;
    Ending:=tmp;
End;

Procedure DoDel;
Var
    a,i:Longint;
    tmp,tmp_:PNode;
Begin
    Readln(a);
    tmp:=Now;
    For i:=1 To a+1 Do
        Begin
            tmp_:=tmp;
            tmp:=tmp^.Next;
            If i>1
            Then Dispose(tmp_);
            If tmp=nil
            Then Break;
        End;
    now^.Next:=tmp;
    If tmp=nil
    Then Ending:=now;
End;

Procedure DoMove;
Var
    a,i:Longint;
Begin
    Readln(a);
    If a=0
    Then Now:=Head
    Else For i:=1 To a Do
        If Now^.Next<>nil
        Then Now:=Now^.Next
        Else Break;
End;

Procedure DoPrint;
Begin
    Writeln(Now^.Next^.Data);
End;

```

```

Procedure Main;
Var
  Command:String;
Begin
  While not Eof Do
  Begin
    Readln(Command);
    If Command='ADD'
    Then DoAdd;
    If Command='DEL'
    Then DoDel;
    If Command='MOVE'
    Then DoMove;
    If Command='PRINT'
    Then DoPrint;
  End;
End;

Procedure Done;
Begin
  Close(Output);
End;

Begin
  Init;
  Main;
  Done;
End.

```

Thuật toán đã tối ưu.

```

Program Links;

Type
  PNode=^TNode;
  TNode=Record
    Data:String;
    Next:PNode;
  End;

Var
  Head,Now,Ending:PNode;
  Re_Head,Re_End:PNode;

Procedure Init;
Begin
  Assign(Input,'line.in');
  Reset(Input);
  Assign(Output,'line.out');
  Rewrite(Output);
  New(Head);
  FillChar(Head^,SizeOf(Head^),0);
  Now:=Head;
  Ending:=Head;

```

```

    New(Re_Head);
    FillChar(Re_Head^,SizeOf(Re_Head^),0);
    Re_End:=Re_Head;
End;

Procedure DoAdd;
Var
    tmp:PNode;
    st:String;
Begin
    Readln(st);
    If Re_Head^.Next=nil
    Then New(tmp)
    Else Begin
        tmp:=Re_Head^.Next;
        Re_Head^.Next:=Re_Head^.Next^.Next;
    End;
    tmp^.Data:=st;
    Ending^.Next:=tmp;
    tmp^.Next:=nil;
    Ending:=tmp;
End;

```

```

Procedure DoDel;
Var
    a,i:Longint;
    tmp:PNode;
Begin
    Readln(a);
    tmp:=Now;
    For i:=1 To a Do
    Begin
        tmp:=tmp^.Next;
        If tmp=nil
        Then Break;
    End;
    If Re_Head^.Next=nil
    Then Re_End:=Re_Head;
    Re_End^.Next:=now^.Next;
    Re_End:=tmp;
    now^.Next:=tmp^.Next;
    If tmp^.Next=nil
    Then Ending:=now;
    tmp^.Next:=nil;
End;

```

```

Procedure DoMove;
Var
    a,i:Longint;
Begin
    Readln(a);
    If a=0
    Then Now:=Head
    Else For i:=1 To a Do

```

```

        If Now^.Next<>nil
        Then Now:=Now^.Next
        Else Break;
End;

Procedure DoPrint;
Begin
    Writeln(Now^.Next^.Data);
End;

Procedure Main;
Var
    Command:String;
Begin
    While not Eof Do
    Begin
        Readln(Command);
        If Command='ADD'
        Then DoAdd;
        If Command='DEL'
        Then DoDel;
        If Command='MOVE'
        Then DoMove;
        If Command='PRINT'
        Then DoPrint;
    End;
End;

Procedure Done;
Begin
    Close(Output);
End;

Begin
    Init;
    Main;
    Done;
End.

```

Chương trình sinh dữ liệu 1.

```

Program Line_Make;

Var
    i,j:Longint;
    st:String;

Begin
    Assign(Output,'line.in');
    Rewrite(Output);
    For i:=1 To 10 Do
    Begin
        For j:=1 To 100000 Do
        Begin

```

```

        Str(j,st);
        Writeln('ADD');
        Writeln(st);
    End;
    Writeln('MOVE');
    Writeln(0);
    Writeln('PRINT');
    For j:=1 To 100000 Do
    Begin
        Writeln('DEL');
        Writeln(1);
    End;
End;
Close(Output);
End.

```

Chương trình sinh dữ liệu 2.

```

Program Line_Make;

Var
    i,j:Longint;
    st:String;

Begin
    Assign(Output,'line.in');
    Rewrite(Output);
    For i:=1 To 10 Do
    Begin
        For j:=1 To 400000 Do
        Begin
            Str(j,st);
            Writeln('ADD');
            Writeln(st);
        End;
        Writeln('MOVE');
        Writeln(0);
        Writeln('DEL');
        Writeln(300000);
        Writeln('PRINT');
        For j:=1 To 100000 Do
        Begin
            Writeln('DEL');
            Writeln(1);
        End;
    End;
    Close(Output);
End.

```

Chương trình sinh dữ liệu 3.

```

Program Line_Make;

Var

```

```

i,j:Longint;
st:String;

Begin
  Assign(Output,'line.in');
  Rewrite(Output);
  For i:=1 To 10 Do
    Begin
      For j:=1 To 400000 Do
        Begin
          Str(j,st);
          Writeln('ADD');
          Writeln(st);
          Writeln('MOVE');
          Writeln(0);
          Writeln('PRINT');
          Writeln('DEL');
          Writeln(1);
        End;
      End;
    Close(Output);
  End.

```

Chương trình sinh dữ liệu 4.

```

Program Line_Make;

Var
  i,j,k,L:Longint;
  st:String;

Begin
  Assign(Output,'line.in');
  Rewrite(Output);
  Randomize;
  k:=0;
  For i:=1 To 10 Do
    Begin
      For j:=1 To 400000 Do
        Begin
          Repeat
            L:=Random(3);
          Until not ((L<>1)and(k=0));
          Case L Of
            0: Begin
              Writeln('MOVE');
              Writeln('0');
              Writeln('DEL');
              Repeat
                L:=Random(10)+1;
              Until L<=k;
              Writeln(L);
              Dec(k,L);
            End;
          End;
        End;
      End;
    End;
  End.

```

```

1: Begin
    Writeln('ADD');
    Writeln(j);
    Inc(k);
End;
2: Begin
    Writeln('MOVE');
    Writeln('0');
    Writeln('PRINT');
End;
End;
End;
End;
Close(Output);
End.

```

Chương trình sinh dữ liệu 5.

```

Program Line_Make;

Var
    i,j,k,L:Longint;
    st:String;

Begin
    Assign(Output,'line.in');
    Rewrite(Output);
    Randomize;
    k:=0;
    For i:=1 To 10 Do
        Begin
            For j:=1 To 800000 Do
                Begin
                    Repeat
                        L:=Random(3);
                    Until not ((L<>1)and(k=0));
                    Case L Of
                        0: Begin
                            Writeln('MOVE');
                            Writeln('0');
                            Writeln('DEL');
                            Repeat
                                L:=Random(10)+1;
                            Until L<=k;
                            Writeln(L);
                            Dec(k,L);
                        End;
                    1: Begin
                        Writeln('ADD');
                        Writeln(j);
                        Inc(k);
                    End;
                    2: Begin
                        Writeln('MOVE');

```

```
        Writeln('0');
        Writeln('PRINT');
    End;
End;
End;
Close(Output);
End.
```